
گزارش فنی پروژه پلتفرم استریم موسیقی

Music Streaming Platform — Technical Report

گروه شماره ۶

اعضای گروه: فاطمه طامهري محمد عقیل حافظی نیما کهدنارویی

تاریخ تهیه: مرداد ۱۴۰۵

فصل ۱

قوانین توسعه و فرآیند آن

به منظور حفظ یکپارچگی کدها، تسهیل کار تیمی و افزایش پایداری سیستم در طول چرخه حیات توسعه نرم افزار، مجموعه استانداردهای مشخصی برای پایگاه کد (Codebase) تدوین شده است. رعایت این قوانین در هر دو لایه فرانتاند و بکاند الزامی بوده و تمامی اعضای تیم موظف به پیروی از آن‌ها در فرآیند توسعه هستند.

۱.۱ قوانین نام‌گذاری برنچ‌ها (Branching Strategy)

پروژه بر اساس مدل مبتنی بر شاخه ویژگی‌ها (Feature Branch Workflow) مدیریت می‌شود. در این رویکرد، شاخه اصلی (main) همواره باید شامل کدهای پایدار و آماده‌ی انتشار باشد. برای هر تغییر، یک شاخه مستقل ایجاد می‌گردد که نام آن باید نمایانگر نوع و هدف تغییرات باشد:

- **feat/**: برای توسعه ویژگی‌ها و ماژول‌های جدید در سیستم.
 - مثال: feat/playlists-page یا feat/music-player-queue
- **fix/**: جهت رفع باگ‌ها و خطاهای گزارش‌شده.
 - مثال: fix/role-guard-redirect
- **refactor/**: برای بازنویسی و بهبود ساختار کدهای موجود بدون تغییر در رفتار ظاهری آن‌ها.
 - مثال: refactor/api-client
- **docs/**: جهت افزودن یا به‌روزرسانی مستندات فنی پروژه.
 - مثال: docs/spec-routing

۲.۱ قوانین نام‌گذاری متغیرها و کلاس‌ها (Naming Conventions)

با توجه به استفاده از معماری‌های متفاوت در سمت کاربر و سرور، قراردادهای نام‌گذاری متناسب با اکوسیستم و زبان برنامه‌نویسی هر لایه به شرح زیر تعریف شده است:

۱.۲.۱ لایه فرانت‌اند (React / TypeScript)

- کامپوننت‌ها (Components) و رابط‌ها (Interfaces): از قالب PascalCase استفاده می‌شود.
 - مثال: کامپوننت PlaylistCard یا رابط UserProfileView
- متغیرها، توابع و هوک‌ها (Variables, Functions, Hooks): از قالب camelCase استفاده می‌شود.
 - مثال: متغیر currentTrack، تابع fetchPlaylists() یا هوک useAuthStore()
- ثابت‌ها (Constants): تماماً با حروف بزرگ و با استفاده از زیرخط در قالب UPPER_SNAKE_CASE نوشته می‌شوند.
 - مثال: SUBSCRIPTION_LIMITS یا ROLES

۲.۲.۱ لایه بک‌اند (Django / Python)

- کلاس‌ها (مانند Models, Views, Serializers): از قالب PascalCase استفاده می‌شود.
 - مثال: مدل SubscriptionPaymentLog یا سریالایزر TrackReadSerializer
- متغیرها، توابع و فیلدهای پایگاه داده: مطابق با قراردادهای استاندارد پایتون (PEP 8) از قالب snake_case استفاده می‌شود.
 - مثال: فیلد stream_count یا تابع get_gateway_amount()
- ثابت‌ها (Constants): مشابه لایه فرانت‌اند، در قالب UPPER_SNAKE_CASE تعریف می‌گردند.
 - مثال: MAX_AUDIO_SIZE_MB

۳.۱ قوانین ثبت تغییرات (Commit Messages)

برای ثبت تغییرات در مخزن کد (Git)، از استاندارد Conventional Commits استفاده می‌شود. پیام‌ها باید کوتاه، با افعال امری (Imperative mood) و به زبان انگلیسی نوشته شوند تا بررسی تاریخچه تغییرات بهینه‌تر گردد. ساختار کلی پیام‌ها به فرم type(scope): subject است:

- feat: برای افزودن ویژگی جدید.

- مثال: feat(playlists): add create playlist modal
- **fix:** برای رفع باگ.
- مثال: fix(player): correct shuffle queue order
- **docs:** برای تغییرات مستندات.
- مثال: docs(spec): document localStorage keys
- **test:** برای افزودن یا اصلاح تست‌ها.
- مثال: test(auth): add unit tests for login payload

۴.۱ فرآیند توسعه و مرور کد (Development & Code Review Process)

چرخه حیات توسعه یک ویژگی نرم‌افزاری در این پروژه به گونه‌ای طراحی شده است که حداقل میزان تداخل کد (Merge Conflict) و بالاترین کیفیت خروجی را تضمین کند. این چرخه شامل گام‌های متوالی زیر است:

۱. دریافت آخرین نسخه کد: پیش از آغاز هرگونه توسعه، برنامه‌نویس باید آخرین تغییرات شاخه اصلی (main) را دریافت کرده (git pull) و همگام‌سازی‌های لازم را انجام دهد.
۲. ایجاد شاخه مستقل: توسعه‌دهنده یک شاخه جدید بر اساس قوانین نام‌گذاری ذکر شده ایجاد می‌نماید (git checkout -b).
۳. توسعه و ارزیابی محلی: کدنویسی در محیط ایزوله انجام می‌گیرد. پیش از اتمام کار، توسعه‌دهنده موظف است کیفیت کدهای خود را توسط ابزارهای خودکار بررسی نماید (اجرای npm run verify در فرانت‌اند برای کامپایل و اجرای Vitest، و اجرای pytest در بک‌اند).
۴. ایجاد درخواست ادغام (Pull Request): پس از ثبت تغییرات (Commit) و ارسال آن‌ها به مخزن راه دور، یک Pull Request (PR) ایجاد می‌گردد. در توضیحات این PR باید خلاصه‌ای از تغییرات، شماره بخش مربوطه در مستندات (مانند 2.7\$)، تصاویر در صورت تغییر رابط کاربری، و چک‌لیست تست‌ها قید شود.
۵. مرور کد (Code Review): درخواست ادغام باید توسط حداقل یک مهندس دیگر در تیم بررسی شود. در این مرحله، انطباق کد با استانداردهای معماری، مسائل امنیتی، و عملکرد صحیح بررسی می‌گردد.
۶. ادغام (Merge): پس از تأیید (Approve) و اطمینان از پاس شدن تمامی تست‌ها، تغییرات به شکل یکپارچه (معمولاً با روش Squash or Merge) وارد شاخه اصلی سیستم می‌گردد.

فصل ۲

ساختار پروژه و توجیه معماری

سیستم توسعه‌یافته برای این پلتفرم استریم موسیقی، بر پایه معماری توزیع‌شده (Decoupled/Headless Architecture) طراحی گردیده است. در این رویکرد، هسته پردازشی سرور (بک‌اند) و رابط کاربری (فرانت‌اند) به عنوان دو پروژه کاملاً مستقل توسعه یافته‌اند. در ادامه، ساختار هر یک از این بخش‌ها و دلایل مهندسی اتخاذ این معماری تشریح شده است.

۱.۲ ساختار پروژه بک‌اند (Backend Architecture)

پروژه بک‌اند با بهره‌گیری از فریم‌ورک Django و افزونه Django REST Framework (DRF) بر پایه زبان پایتون توسعه یافته است. معماری این بخش بر اساس اصول «طراحی مبتنی بر دامنه» (Domain-Driven Design) سازمان‌دهی شده و ساختار آن شامل موارد زیر است:

- **اپلیکیشن‌های تفکیک‌شده (Logical Apps):** پروژه به ماژول‌های منطقی و مستقلی نظیر user (مدیریت کاربران و احراز هویت)، music (مدیریت کاتالوگ و استریم)، payment (تراکنش‌های مالی) و management (پنل‌های مدیریتی) تقسیم شده است.
- **لایه‌ی مدل و نگاشت شیء-رابطه‌ای (Models & ORM):** تعریف دقیق موجودیت‌های پایگاه داده و روابط بین آن‌ها (نظیر آلبوم‌ها، قطعات موسیقی، و گزارش‌های مالی) جهت تعامل بهینه با پایگاه داده.
- **جداسازی منطق تجاری از لایه انتقال (Views & Services):** منطق اصلی کسب‌وکار در فایل‌های مستقل services.py پیاده‌سازی شده و فایل‌های views.py صرفاً مسئولیت دریافت درخواست‌های HTTP و پاسخ‌دهی را بر عهده دارند.
- **نقاط پایانی یکپارچه (RESTful API Endpoints):** طراحی رابط‌های برنامه‌نویسی استاندارد به همراه استفاده از سریالایزرها (Serializers) جهت اعتبارسنجی داده‌های ورودی و تبدیل اشیاء پیچیده

به فرمت JSON برای ارتباط با کلاینت.

۲.۲ ساختار پروژه فرانت‌اند (Frontend Architecture)

پروژه فرانت‌اند به عنوان یک برنامه تک‌صفحه‌ای (SPA) با استفاده از کتابخانه React (نسخه ۱۸)، ابزار ساخت Vite و زبان TypeScript توسعه یافته است. این معماری به صورت خاص برای پاسخگویی به نیازهای یک پلتفرم پخش موسیقی بهینه‌سازی شده است:

- **معماری مبتنی بر مؤلفه (Component-Based Architecture):** تقسیم رابط کاربری به اجزای کوچک، مستقل و قابل استفاده مجدد (در دایرکتوری components). این مؤلفه‌ها به بخش‌های مشترک (common)، پخش‌کننده موسیقی (player) و ویژگی‌های خاص دامنه دسته‌بندی شده‌اند.
- **مدیریت وضعیت سراسری (Global State Management):** استفاده از کتابخانه Zustand جهت مدیریت وضعیت‌های پیچیده و سراسری برنامه. این بخش به‌ویژه برای یکپارچگی وضعیت پخش‌کننده موسیقی در store/playerStore.ts حیاتی است تا با تغییر صفحات، پخش موسیقی قطع نگردد.
- **مسیریابی و کنترل دسترسی (Routing & Role Guards):** پیاده‌سازی مسیریابی سمت کلاینت توسط React Router و استفاده از کامپوننت‌های محافظتی (نظیر RoleGuard.tsx) جهت محدودسازی دسترسی کاربران بر اساس نقش‌های تعریف‌شده (شنونده، هنرمند، پشتیبان، مدیر).
- **مدیریت منابع و لایه ارتباطی (Assets & API Layer):** ایزوله کردن منطق ارتباط با سرور در دایرکتوری lib/api. این ساختار امکان جابه‌جایی یکپارچه از داده‌های شبیه‌سازی‌شده (Mock Services در فاز اول) به فراخوانی‌های واقعی API (در فاز دوم) را بدون تغییر در لایه رابط کاربری فراهم آورده است.

۳.۲ توجیه معماری و رویکرد جداسازی (Architectural Justification)

انتخاب معماری کلاینت-سرور تفکیک‌شده به جای استفاده از نرم‌افزارهای یکپارچه سنتی (Monolithic)، بر پایه چندین اصل بنیادین مهندسی نرم‌افزار صورت گرفته است:

- **اصل تفکیک وظایف (Separation of Concerns):** با این جداسازی، تیم توسعه بک‌اند می‌تواند منحصراً بر امنیت، پردازش‌های سنگین (مانند استخراج ویژگی‌های صوتی توسط یادگیری ماشین) و مدیریت داده تمرکز کند. در مقابل، تیم فرانت‌اند تمام منابع خود را به بهبود تجربه کاربری (UX)، انیمیشن‌ها و مدیریت وضعیت پلیر اختصاص می‌دهد.
- **پایداری تجربه کاربری در استریم موسیقی:** در یک پلتفرم موسیقی، بارگذاری مجدد صفحات در

رویکردهای یکپارچه منجر به قطع شدن صدای در حال پخش می‌شود. جداسازی فرانت‌اند و استفاده از معماری SPA تضمین می‌کند که پلیر صوتی همواره در پس‌زمینه فعال بماند.

- **مقیاس‌پذیری و استقرار مستقل (Scalability & Independent Deployment):** دو پروژه می‌توانند چرخه‌های انتشار مستقلی داشته باشند و در کانتینرهای مجزای داکر (Docker Containers) مستقر شوند؛ در صورت افزایش ترافیک، می‌توان منابع پردازشی سرور را بدون نیاز به تغییر در منابع کلاینت استاتیک، مقیاس‌دهی کرد.
- **توسعه‌پذیری و رویکرد مبتنی بر رابط (API-First Approach):** با طراحی سیستم به گونه‌ای که تمامی تعاملات از طریق REST API انجام شود، توسعه اپلیکیشن‌های بومی موبایل (iOS و Android) یا برنامه‌های دسکتاپ در آینده بدون نیاز به نوشتن مجدد منطق بک‌اند امکان‌پذیر خواهد بود.

فصل ۳

توجیه نگهداری پذیری کد

کدبیس (Codebase) این پلتفرم با رویکردی آینده‌نگرانه و مبتنی بر استانداردهای روز مهندسی نرم‌افزار توسعه یافته است. در طراحی و پیاده‌سازی این سیستم، تمرکز ویژه‌ای بر قابلیت توسعه‌پذیری (Scalability)، پایداری و کاهش هزینه‌های نگهداری (Maintainability) در چرخه‌های آتی حیات نرم‌افزار شده است.

۱.۳ خوانایی و تمیزی کد (Clean Code & Readability)

یکی از ارکان اصلی نگهداری‌پذیری، قابلیت درک سریع کد توسط سایر توسعه‌دهندگان است. در این پروژه، خوانایی از طریق نام‌گذاری‌های معناگرا، تعیین نوع داده‌ها (Type Safety) و مستندسازی درون‌کدی (Inline Documentation) تضمین شده است:

- استفاده از تایپ‌اسکرپت و مدل‌های صریح: در فرانت‌اند، تمامی موجودیت‌های دامنه (مانند User، Playlist، Track) در دایرکتوری مجزای src/types به صورت صریح تعریف شده‌اند. این امر از بروز خطاهای زمان اجرا جلوگیری می‌کند.
- نام‌گذاری توصیفی و معناگرا: توابع و متغیرها دقیقاً بیانگر عملکرد خود هستند. به عنوان مثال، در فرانت‌اند از توابعی نظیر handleSubscriptionUpgrade() و در بک‌اند از متدهایی مانند get_effective_subscription_tier() استفاده شده که نیاز به کامنت‌گذاری اضافی را مرتفع می‌سازد.
- مستندسازی منطق‌های پیچیده: در بخش‌هایی که پردازش‌های سنگین صورت می‌گیرد (نظیر audio_features.py یا recommender.py)، از Docstring‌های ساختاریافته و کامنت‌های خطی برای توضیح الگوریتم‌ها استفاده شده است.

۲.۳ ماژولار بودن و استفاده مجدد (Modularity & Reusability)

سیستم به بلوک‌های کوچک و مستقلی تقسیم شده است که قابلیت استفاده مجدد را به حداکثر می‌رسانند:

- کامپوننت‌های مشترک در رابط کاربری: عناصر تکرارپذیر رابط کاربری به صورت کامپوننت‌های عمومی در دایرکتوری components/common توسعه یافته‌اند. کامپوننت‌های EmptyState و PageHeader در صفحات متعددی بدون تکرار کدهای HTML/CSS بازتولید شده‌اند.
- توابع کمکی و اعتبارسنج‌ها (Validators): در لایه بک‌اند، منطق اعتبارسنجی فایل‌ها در validators.py (شامل توابعی چون validate_audio_file و validate_image_size) کپسوله شده و به عنوان پارامتر به سریالایزرهای مختلف پاس داده می‌شوند.

۳.۳ اصل تفکیک وظایف (Separation of Concerns)

این اصل به‌طور سخت‌گیرانه‌ای در سراسر معماری کلاینت و سرور رعایت شده است:

- تفکیک لایه شبکه از رابط کاربری: در فرانت‌اند، هیچ‌گونه فراخوانی مستقیم API در داخل کامپوننت‌های React انجام نمی‌شود. تمامی درخواست‌های شبکه به توابع مستقلی در دایرکتوری lib/api/ محول شده‌اند.
- تفکیک وضعیت (State) از منطق نمایش: منطق پخش موسیقی و وضعیت صف پخش به‌طور کامل از کامپوننت‌های بصری جدا شده و در store/playerStore.ts (با استفاده از Zustand) مدیریت می‌گردد.
- معماری Service-Layer در بک‌اند: فایل‌های views.py صرفاً وظیفه دریافت درخواست HTTP و اعتبارسنجی اولیه را دارند. منطق اصلی کسب‌وکار و تعاملات دیتابیس به services.py منتقل شده که از حاشیه‌نویسی @transaction.atomic برای حفظ یکپارچگی تراکنش‌های پایگاه داده استفاده می‌کند.

۴.۳ اعمال اصول مهندسی نرم‌افزار (SOLID و DRY)

- رعایت اصل عدم تکرار (DRY — Don't Repeat Yourself): متون ثابت در فایل‌های دیکشنری متمرکز نظیر lib/constants/appText.ts تعریف شده‌اند. مدیریت سطوح دسترسی کاربران نیز به جای بررسی در هر صفحه، در کامپوننت واحد RoleGuard.tsx متمرکز شده است.
- اصل تک‌مسئولیتی (SRP — Single Responsibility Principle): هر فایل یا ماژول صرفاً یک دلیل برای تغییر دارد. ماژول payment/services.py منحصرأً وظیفه مدیریت ارتباط با درگاه پرداخت

(زرین‌پال) را بر عهده دارد و کاملاً از سیستم مدیریت اشتراک کاربران جداست. این طراحی تضمین می‌کند که در صورت تغییر درگاه پرداخت، نیازی به تغییر در ماژول‌های دیگر نخواهد بود.

فصل ۴

مدل‌های بک‌اند، روابط و توجیه آن‌ها

پایگاه داده این پلتفرم با بهره‌گیری از قابلیت‌های نگاشت شی-رابطه‌ای (ORM) فریم‌ورک جنگو و بر پایه اصول بنیادین طراحی پایگاه داده‌های رابطه‌ای (RDBMS) توسعه یافته است. در این معماری، تلاش شده است تا با استفاده از الگوهای استاندارد مدل‌سازی، موجودیت‌های سیستم به شکلی نرمال‌سازی‌شده و بهینه جهت پردازش حجم بالای داده‌های استریمینگ سازمان‌دهی شوند.

۱.۴ معرفی مدل‌های اصلی (Core Models)

هسته اصلی داده‌های این پلتفرم در قالب چندین مدل (Model) مجزا در اپلیکیشن‌های مختلف جنگو تعریف شده است. مهم‌ترین این مدل‌ها عبارتند از:

- **مدل User:** با ارث‌بری از `AbstractUser` جنگو، به عنوان موجودیت پایه برای تمام کاربران سیستم عمل می‌کند. شامل فیلدهای هویتی نظیر `email`، `display_name`، نقش کاربر (`role`)، سطح اشتراک (`subscription_tier`) و تعداد استریم‌های روزانه (`streamed_today`) است.
- **مدل Artist:** نمایانگر هنرمندان در سیستم است. علاوه بر ارث‌بری از کلاس `User`، حاوی داده‌های اختصاصی هنرمندان نظیر نام هنری (`stage_name`)، لینک‌های نمونه کار (`portfolio_links`)، وضعیت احراز هویت (`verification_status`) و آمار تجمیعی پخش قطعات (`total_streams`) است.
- **مدل Track:** اصلی‌ترین موجودیت در اپلیکیشن موسیقی است که اطلاعات هر قطعه را ذخیره می‌کند. شامل فایل صوتی (`audio_file`)، مدت زمان (`duration_seconds`)، آمار پخش (`stream_count`) و یک `JSONField` با نام `audio_features` برای نگهداری بردارهای استخراج‌شده توسط هوش مصنوعی است.
- **مدل Album:** نمایانگر مجموعه‌ای از قطعات موسیقی متعلق به یک هنرمند است و فیلدهایی برای عنوان (`title`)، سال انتشار (`release_year`) و کاور آلبوم (`cover_art`) دارد.

- **مدل Playlist:** برای نگهداری لیست پخش‌های ساخته‌شده توسط کاربران طراحی شده و شامل عنوان (name) و تصویر کاور است.
- **مدل StreamEvent:** یک موجودیت لاگ‌گونه است که هر بار پخش یک قطعه توسط یک کاربر را با ذکر زمان دقیق (created_at) ثبت می‌کند.

۲.۴ روابط بین مدل‌ها (Relationships)

۱.۲.۴ روابط یک‌به‌چند (One-to-Many)

- هر هنرمند (Artist) می‌تواند خالق چندین قطعه (Track) و چندین آلبوم (Album) باشد، اما هر قطعه یا آلبوم متعلق به یک هنرمند اصلی است (استفاده از ForeignKey در مدل‌های Track و Album به Artist).
- هر لیست پخش (Playlist) منحصرأ متعلق به یک کاربر (User) است، اما یک کاربر می‌تواند ده‌ها لیست پخش ایجاد کند.
- هر رویداد پخش (StreamEvent) به یک کاربر خاص و یک قطعه خاص متصل است.

۲.۲.۴ روابط چندبه‌چند (Many-to-Many)

- **قطعات و لیست پخش‌ها:** یک قطعه موسیقی می‌تواند در چندین لیست پخش حضور داشته باشد و یک لیست پخش شامل چندین قطعه است. از یک مدل واسطه (Through Model) با نام PlaylistTrack استفاده شده تا ترتیب هر آهنگ از طریق فیلد position ذخیره گردد.
- **شبکه اجتماعی کاربران:** رابطه دنبال کردن کاربران (Follow) با استفاده از ManyToManyField به نام following در مدل User (رابطه با خود) پیاده‌سازی شده است.

۳.۲.۴ روابط یک‌به‌یک (One-to-One)

- تنظیمات هر کاربر در مدل Preferences نگهداری می‌شود که رابطه‌ای یک‌به‌یک با مدل User دارد تا تنظیمات ظاهری (مثل theme و language) از داده‌های هویتی تفکیک شوند.

۳.۴ توجیه طراحی و نرمال‌سازی داده‌ها (Architectural Justification)

- **جداسازی هنرمند از طریق وراثت چندجدولی (Multi-Table Inheritance):** به جای افزودن فیلدهای مربوط به هنرمندان به مدل پایه User، مدل Artist به صورت مجزا ارث‌بری کرده است. این

کار از ایجاد رکوردهای تُنک (Sparse Tables) و پر شدن پایگاه داده با مقادیر NULL برای کاربران عادی جلوگیری می‌کند.

- **جلوگیری از افزونگی داده‌ها (Data Redundancy):** نام هنرمند یا آلبوم به صورت متنی درون مدل Track ذخیره نشده است. با ارجاع از طریق کلید خارجی (Foreign Key)، تغییر نام هنری یک هنرمند تنها در یک رکورد اعمال می‌شود.
- **استفاده از JSONField برای ویژگی‌های صوتی:** هر قطعه موسیقی شامل ده‌ها بردار و ویژگی چندبعدی (مانند clap_embedding یا tonnetz) است. استفاده از JSONField در مدل Track یک راه‌حل مهندسی و شبه-NoSQL درون یک پایگاه داده رابطه‌ای است که سرعت استخراج داده‌ها توسط سرویس recommender.py را به طرز چشمگیری افزایش می‌دهد.
- **مدل‌سازی رویدادمحور برای استریم‌ها:** وجود مدل StreamEvent دو مزیت بزرگ دارد: نخست، محاسبه دقیق «شنوندگان یکتا» (Unique Listeners) در برابر «تعداد کل پخش‌ها» (Total Streams) برای سیستم حسابرسی (MonthlyArtistAudit)؛ دوم، ایجاد قابلیت کنترل سوءاستفاده (Abuse Prevention) و اعمال محدودیت‌های روزانه برای کاربران پایه.

فصل ۵

رعایت قواعد REST

در معماری سرویس‌گرای این پلتفرم استریم موسیقی، طراحی واسط‌های برنامه‌نویسی اپلیکیشن (API) به صورت کامل بر پایه اصول معماری **REST (Representational State Transfer)** صورت پذیرفته است. این انطباق با استانداردها، علاوه بر ایجاد یکپارچگی در تعاملات کلاینت و سرور، قابلیت پیش‌بینی‌پذیری و توسعه‌پذیری سرویس‌ها را برای یکپارچه‌سازی‌های آتی تضمین می‌نماید.

۱.۵ طراحی آدرس‌ها مبتنی بر منابع (Resource-Oriented URLs)

یکی از اصول بنیادین REST، شناسایی موجودیت‌های سیستم به عنوان «منابع» (Resources) و استفاده از اسامی جمع (Plural Nouns) به جای افعال در ساختار مسیرهاست. در این پروژه، مسیرهای تعریف‌شده این الگو را به دقت پیاده‌سازی کرده‌اند:

- **مدیریت لیست‌پخش‌ها:** مسیر `/api/v1/music/playlists/` به صورت یک منبع جمع طراحی شده است. برای دسترسی به یک منبع خاص، از شناسه آن در آدرس استفاده می‌شود (نظیر `./playlists/{id}/`).
- **مدیریت آلبوم‌ها و قطعات:** مسیرهایی مانند `/api/v1/music/albums/` و `/api/v1/music/artist/releases/` دقیقاً به موجودیت آلبوم‌ها و آثار منتشرشده یک هنرمند اشاره دارند.
- **عملیات خاص (Sub-resources):** از الگوهای استاندارد نظیر `/api/v1/music/tracks/{id}/play/` استفاده شده که رویداد پخش یک قطعه مشخص را به عنوان یک زیرمنبع مدیریت می‌کند.

۲.۵ استفاده استاندارد از متدهای HTTP

رابط‌های این پلتفرم به گونه‌ای طراحی شده‌اند که عملیات چهارگانه (CRUD) دقیقاً با متدهای استاندارد پروتکل HTTP مطابقت داشته باشند:

- **متد GET:** صرفاً برای خواندن و واکنشی داده‌ها استفاده می‌شود و هیچ‌گونه تغییر وضعیتی (Side Effect) در سرور ایجاد نمی‌کند.
- **متد POST:** برای ایجاد منابع جدید کاربرد دارد. در `ArtistReleaseViewSet`، ارسال درخواست POST همراه با فایل صوتی و متادیتا، منجر به خلق یک اثر جدید در پایگاه داده می‌گردد.
- **متد PUT و PATCH:** برای به‌روزرسانی منابع استفاده شده‌اند. متد PATCH اجازه می‌دهد تنها بخشی از فیلدهای یک قطعه بدون نیاز به ارسال کل شیء به‌روزرسانی شود.
- **متد DELETE:** منحصراً برای حذف منابع استفاده می‌گردد و باعث حذف قطعه و پاکسازی هوشمند رکوردهای وابسته (مانند آلبوم‌های خالی شده) می‌گردد.

۳.۵ کدهای وضعیت HTTP (Standard Status Codes)

سرور بک‌اند به گونه‌ای برنامه‌ریزی شده است که نتیجه هر درخواست را با ارسال کدهای وضعیت استاندارد HTTP به کلاینت اطلاع‌رسانی کند:

- **200 OK:** برای درخواست‌های موفق `GET`، `PATCH` و `DELETE`.
- **201 Created:** هنگام ایجاد موفقیت‌آمیز یک منبع جدید، نظیر ساخت پلی‌لیست یا انتشار یک قطعه.
- **204 No Content:** در پاسخ به عملیات حذف موفق که نیازی به بازگشت داده‌ای نیست.
- **400 Bad Request:** هنگام نامعتبر بودن داده‌های ورودی، مانند بروز خطای `ValidationError` در `PublishReleaseSerializer`.
- **403 Forbidden:** زمانی که کاربر احراز هویت شده اما دسترسی لازم را ندارد، مثلاً کاربری که قصد ایجاد پلی‌لیست بیش از حد مجاز اشتراک خود را دارد.
- **404 Not Found:** هنگامی که منبع درخواستی (مانند یک آلبوم یا لیست‌پخش خاص) در پایگاه داده موجود نباشد.

۴.۵ معماری بدون حالت و فرمت داده‌ها (Statelessness & Data Format)

مطابق با پارادایم REST، سرویس توسعه‌یافته کاملاً بدون حالت (Stateless) است:

- **احراز هویت توکنی:** شناسایی و اعتبارسنجی کاربران از طریق تبادل توکن‌های JWT در هدر درخواست‌ها (Authorization: Bearer) صورت می‌پذیرد.
- **استانداردسازی فرمت تبادل:** تمام تبادلات داده‌ای در قالب استاندارد **JSON** انجام می‌پذیرد. تنها استثنا، فرم‌های حاوی فایل (نظیر آپلود تصویر کاور یا فایل صوتی) است که با **multipart/form-data** مدیریت می‌شوند، اما پاسخ سرور به این درخواست‌ها نیز در قالب **JSON** خواهد بود.

فصل ۶

الگوهای طراحی (Design Patterns)

یکی از شاخصه‌های اصلی مهندسی نرم‌افزار در سیستم‌های مقیاس‌پذیر، بهره‌گیری از الگوهای طراحی (Design Patterns) اثبات‌شده است. در معماری این پلتفرم، الگوهای طراحی متعددی در هر دو لایه کلاینت و سرور به کار گرفته شده‌اند تا خوانایی، نگهداری‌پذیری، و قابلیت توسعه سیستم تضمین گردد.

۱.۶ الگوهای طراحی در لایه فرانت‌اند (Frontend Patterns)

- **الگوی تامین‌کننده (Provider Pattern):** برای حل مشکل Prop Drilling در برنامه‌های پیچیده‌ای مانند پخش‌کننده موسیقی، از این الگو بهره گرفته شده است. استفاده از `LanguageContext.Provider` و `ThemeModeContext.Provider` در فایل `App.tsx` برای مدیریت سراسری زبان و پوسته برنامه، و کتابخانه Zustand (در `authStore.ts` و `playerStore.ts`) به عنوان پیاده‌سازی مدرن این الگو که وضعیت پخش موسیقی را به صورت سراسری مدیریت می‌کند.
- **الگوی تفکیک کامپوننت‌های نمایشی و منطقی (Container / Presentational Pattern):** کامپوننت‌ها به دو دسته کانتینر (مدیریت‌کننده داده و منطق) و نمایشی (مسئول رندر UI) تقسیم شده‌اند. `PlaylistsPage.tsx` یک کامپوننت کانتینر است که وظیفه فراخوانی API و مدیریت وضعیت‌های بارگذاری را دارد، در مقابل `PlaylistListCard.tsx` یک کامپوننت نمایشی صرف است.
- **الگوی هوک‌های سفارشی (Custom Hooks Pattern):** منطق‌های پیچیده و تکرارپذیر مرتبط با چرخه حیات React درون هوک‌های سفارشی کپسوله شده‌اند. فایل `useIsMobile.ts` منطق مربوط به تشخیص ابعاد صفحه نمایش را در یک هوک سفارشی قرار داده است تا کامپوننت‌های چیدمان با فراخوانی ساده `const isMobile = useIsMobile()` رفتار ریسپانسیو خود را تنظیم کنند.

۲.۶ الگوهای طراحی در لایه بک‌اند (Backend Patterns)

- **الگوی معماری پایه (MTV / MVC Pattern):** معماری پیش‌فرض جنگو یعنی Model-Template-View اصل تفکیک وظایف را در سطح کلان پیاده‌سازی می‌کند. داده‌ها در لایه مدل (مانند Track در music/models.py) تعریف شده‌اند. لایه کنترلر (ArtistReleaseViewSet) درخواست‌های HTTP را مسيردهی می‌کند و لایه نمایش با سریالایزرهاى DRF جایگزین شده است.
- **الگوی لایه سرویس و مفهوم ناظر (Service Layer & Observer Concept):** برای جلوگیری از وابستگی مستقیم ماژول‌ها، از لایه سرویس در بستر الگوی ناظر استفاده شده است. هنگام تایید یک هنرمند در management/services.py (تابع approve_verification)، ماژول مدیریت به صورت ماژولار تابع notify_user را فراخوانی می‌کند. این رویکرد رویدادمحور، وابستگی زیرسیستم اعلانات از زیرسیستم مدیریت هنرمندان را برطرف کرده است.
- **الگوی مدیریت داده‌ها (Active Record & Repository Pattern):** جنگو از الگوی Active Record در ORM استفاده می‌کند. برای جلوگیری از پدیده «کنترلرهای متورم» (Fat Views)، الگوی Repository/Service روی آن پیاده‌سازی شده است. در views.py هیچ‌گونه کوئری مستقیم نوشته نشده و تمام عملیات پایگاه داده در توابعی مستقل در services.py (مانند services.publish_release یا services.record_play) کپسوله شده‌اند.